```
!nvidia-smi
```

```
Thu May  7 18:48:42 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   40C    P8              9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
%%writefile matrix_mul.cu

#include <stdio.h>

#define N 4

__global__ void matrixMul(int A[N][N], int B[N][N], int C[N][N]) {
    int row = threadIdx.y;
    int col = threadIdx.x;

    int sum = 0;

    for (int k = 0; k < N; k++) {
        sum += A[row][k] * B[k][col];
    }

    C[row][col] = sum;
}

int main() {
    int A[N][N], B[N][N], C[N][N];

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = i + j;
            B[i][j] = i * j;
        }
    }

    int (*d_A)[N], (*d_B)[N], (*d_C)[N];

    cudaMalloc((void**)&d_A, sizeof(int) * N * N);
    cudaMalloc((void**)&d_B, sizeof(int) * N * N);
    cudaMalloc((void**)&d_C, sizeof(int) * N * N);

    cudaMemcpy(d_A, A, sizeof(int) * N * N, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, B, sizeof(int) * N * N, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(N, N);

    matrixMul<<<1, threadsPerBlock>>>(d_A, d_B, d_C);

    cudaMemcpy(C, d_C, sizeof(int) * N * N, cudaMemcpyDeviceToHost);

    printf("Result Matrix:\\n");

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

```
Writing matrix_mul.cu
```

```
!nvcc matrix_mul.cu -o matrix_mul
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a future
```

```
!./matrix_mul
```

```
Result Matrix:\n0 14 28 42 \n0 20 40 60 \n0 26 52 78 \n0 32 64 96 \n
```

Start coding or generate with AI.